

C200 PROGRAMMING ASSIGNMENT № 8

Dr. M.M. Dalkilic

Computer Science

School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

November 9, 2024

Please please start early on this HW. The HW is due on **Saturday, November, 16, 2024 at 11:00 PM EST**. Please commit, push and submit your work to the Autograder before the deadline.

1. Make sure that you are **following the instructions** in the PDF, especially the format of output returned by the functions. For example, if a function is expected to return a numerical value, then make sure that a numerical value is returned (not a list or a dictionary). Similarly, if a list is expected to be returned then return a list (not a tuple, set or dictionary).
2. Test **debug** the code well (syntax, logical and implementation errors), before submitting to the Autograder. These errors can be easily fixed by running the code in VSC and watching for unexpected behavior such as, program failing with syntax error or not returning correct output.
3. Make sure that the **code does not have infinite loop (that never exits) or an endless recursion (that never completes)** before submitting to the Autograder. You can easily check for this by running in VSC and watching for program output, if it terminates timely or not.
4. Given that you already tried points 1-3, if you see that Autograder does not do anything (after you press 'submit') and waited for a while (30 seconds to 50 seconds), try refreshing the page or using a different browser.
5. Once you are done testing your code, comment out the tests i.e. the code under the `__name__ == "__main__"` section.

Problem 1: Ksumsort

In this problem you'll be given a non-empty list of numbers `lst` and a non-negative integer $k = 0, 1, 2, \dots$. You'll implement a function `ksumsort(lst, k)` that first builds a list of tuples `[t1, t2, ..., tn]` where $n = 2^{\text{len}(lst)}$. Each tuple is a pair `(s, [x0, ...])` such that $s = \text{sum}([x0, \dots])$ representing all the possible sums. Second, the result is sorted on the absolute difference between k and the sum. The function should generate all non-empty subsets of a list, and construct the tuples of the subset sums and subsets. These tuples should then be sorted based on their absolute difference from k . The sorted list of tuples should then be returned. As example is shown below,

```
1 lst = [1,2,3]
2 print(kns(lst,0))
3 print(kns(lst,3))
4 print(kns(lst,sum(lst)))
```

produces:

```
1 [(1, [1]), (2, [2]), (3, [3]), (3, [1, 2]), (4, [1, 3]), (5, [2, 3]), (6, [1, 2, 3])]
2 [(3, [3]), (3, [1, 2]), (2, [2]), (4, [1, 3]), (5, [2, 3]), (1, [1]), (6, [1, 2, 3])]
3 [(6, [1, 2, 3]), (5, [2, 3]), (4, [1, 3]), (3, [3]), (3, [1, 2]), (2, [2]), (1, [1])]
```

- The list `[1,2,3]` has partial sums `[1],[2],[3],[1,2],[1,3],[2,3],[1,2,3]`.
- The tuples are `[(1, [1]), (2, [2]), (3, [3]), (3, [1, 2]), (4, [1, 3]), (5, [2, 3]), (6, [1, 2, 3])]`.
 - if sorted by $k=0$, then the order is `[(1, [1]), (2, [2]), (3, [3]), (3, [1, 2]), (4, [1, 3]), (5, [2, 3]), (6, [1, 2, 3])]`.
 - if sorted by $k = 3$, then you see that `[(3, [3]), (3, [1, 2])]` come first (the order doesn't matter, because $|3 - 3| = 0$; `(2, [2])`, `(4, [1, 3])` comes next because $|3-2| = |3-4| = 1$; then `(5,[2,3])`, because $|3 - 5| = 2$ then `(6,[1,2,3])` last.
 - If $k = 6$, then the list is reversed
- For any two or more tuples with the same sum, their relative order doesn't matter.

Here is anothe run:

```
1 print(kns([1,2,1],2))
```

giving

1 [(2, [2]), (2, [1, 1]), (1, [1]), (3, [2, 1]), (1, [1]), (3, [1, 2]), ←
(4, [1, 2, 1])]

Programming Problem 1: Ksumsort

- Implement the function (assume at least one number)
- You cannot use any modules for this; however, you are free to use the `list.sort()` method from the lecture.
- A hint: counting in binary might prove useful – you can use `bin()` for that.

Problem 2: Recursion

While equation 1,2 seem to be very complex, you have all the tools you need: `c_2()`, `sigma` as a range function. The `c_2()` function is provided in the starter code to assist you in this problem. You are explicitly allowed (and encouraged) to use both `sum()` and list comprehension.

$$B_0 = 1 \quad (1)$$

$$B_n = \frac{-\sum_{k=0}^{n-1} \binom{n+1}{k} B_k}{n+1} \quad (2)$$

$$B(1) = \frac{-\sum_{k=0}^0 \binom{2}{k} B_0}{1+1} \quad (3)$$

$$= -\frac{\binom{2}{0} 1}{2} \quad (4)$$

$$= -.5 \quad (5)$$

$$B(2) = -\frac{[\sum_{k=0}^1 \binom{3}{k} B_k]}{3} \quad (6)$$

$$= -\frac{[\binom{3}{0} B_0 + \binom{3}{1} B_1]}{3} \quad (7)$$

$$= -\frac{[1(1) + 3(-.5)]}{3} \quad (8)$$

$$= -\frac{[1 - 1.5]}{3} = .5/3 = .1\bar{6}$$

$$B(3) = -\frac{[\sum_{k=0}^2 \binom{4}{k} B_k]}{4} \quad (9)$$

$$= -\frac{[\binom{4}{0} B_0 + \binom{4}{1} B_1 + \binom{4}{2} B_2]}{4} \quad (10)$$

$$= -\frac{[1(1) + 4(-.5) + 6(.1\bar{6})]}{4} \quad (11)$$

$$= -\frac{[1 - 2 + 1]}{4} = 0 \quad (12)$$

Here are some outputs:

output

```

1 B(0) = 1
2 B(1) = -0.5
3 B(2) = 0.16666666666666666
4 B(3) = -0.0
5 B(4) = -0.03333333333333333
6 B(5) = -7.401486830834377e-17

```

Programming Problem 2: Recursion

- Complete the function `B()` using recursion.
- You can (and are encouraged to) use `sum()`.
- List comprehension is useful too!

Problem 3: Candlesticks

Given a sequence of values $[v_0, v_1, \dots, v_n]$ where $0, 1, \dots, n$ represents time, a candlestick is the most popular visualization that is used to both summarize and inform financial analysts. The colors, red and green, indicate whether the valuation has negatively or positively changed, respectively. The line gives the range of valuations. The rectangle's height indicates the starting and ending value for that time period.

For a given sequence, a rectangle and lines are drawn as shown in Fig. 1 (right).

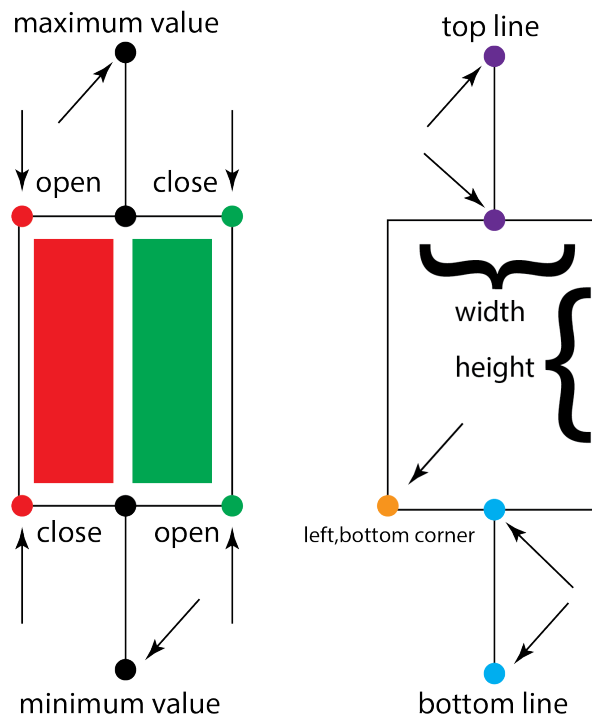


Figure 1: A candlestick image (right). The rectangle is created by the opening and closing values. The top line is from the maximal value to the top middle of the box. The bottom line is from the minimal value to the bottom middle of the rectangle. The color is determined by the open, close values. If open is greater than close, then the color is red; otherwise it's green. The rectangle is determined by the lower-left point (x, y), width, and height. The two lines are determined by two points computed from max, min, open, and close. For plotting, we use matplotlib patches.

For any sequence we can quickly get the four values we need:

```
1 >>> import random as rn
2 >>> prices = [rn.randint(2,50) for _ in range(200)]
3 >>> open,close,max_p,min_p = prices[0],prices[len(prices)-1],max(←
    prices),min(prices)
4 >>> open,close,max_p,min_p
5 (37, 2, 50, 2)
```

Using matplotlib's patches https://matplotlib.org/stable/api/_as_gen/matplotlib.patches.

Rectangle.html we can translate the four values into a candlestick, Fig. 1 (left). To plot a rectangle, we'll need the lower-lefthand corner (x,y), width, and height.

Note: The input is a collection of values contained inside a list, where each sublist represents a candlestick. You will iterate over the contents of the list one-by-one and process each sublist inside the make() function to plot the candlesticks. The plotting is taken care of by the matplotlib patches but you still need to finish the make() function to return the values that are needed by the matplotlib patches. For quickly understanding how it works, see the starter tests for this problem under "__main__".

Hint: As each sublist is passed inside the make() function, you will first find the correct values of open, close, max_p and min_p (from the sublist), and then use those values to calculate the values required to plot the candlestick. How to find, open, close, max_p and min_p is shown in the code listing above. The open and close will be used as the y values (i.e. the values on the y axis) and you have to find them from the sublist.

Let's look at the code:

```
1 data = [[20,15,32,10],[10,14,15,9],
2         [22,23,27,9],[15,16,16,15],
3         [26,12,30,2],[5,30,40,4]]
4
5 # open, close, max_p, min_p = 20, 15, 32, 10
6
7 # INPUTS ith candle, starting value of x, default width, and the four ↵
   critical values: open, close, max\_p, min\_p.
8 # RETURN three tuples: (point, width, height, color), topline, ↵
   bottomline
9 # point is the coordinates of the lower-left point x, y, width and ↵
   height are numeric values and color will be a string of color ex. '↵
   red' or 'green'
10 # topline ((xt0,yt0),(xt1,yt1)) line from max to top middle of ↵
    rectangle
11 # bottomline ((xb0,yb0),(xb1,yb1)) line from min to bottom middle of ↵
    rectangle
12
13 # When you see the testing code under __main__, you will see that the ↵
    first three values of the first tuple i.e., point, width and height↵
    are passed as the first argument of matplotlib.patches.Rectangle↵
    () function and the last value i.e. color is passed as the second ↵
    argument. Feel free to play around with the test code to get a ↵
    feeling of how it is working. You will understand it much better ↵
    with a bit of experimentation.
```

14

```

15 def make(i, start, width_default, d):
16     pass
17
18 fig = plt.figure()
19 ax = fig.add_subplot(111)
20 start = 0
21 default_width = 10
22
23 for i in range(len(data)):
24
25     candle_box, top_line, bottom_line = make(i, start, default_width, ←
        data[i])
26     print(candle_box)
27     ax.add_patch(matplotlib.patches.Rectangle(*candle_box[0:3], color =←
        candle_box[3]))
28     plt.plot([x for x, _ in top_line], [y for _, y in top_line], 'black')
29     plt.plot([x for x, _ in bottom_line], [y for _, y in bottom_line], '←
        black')
30     start += default_width
31
32 plt.xlabel("time (hour)")
33 plt.ylabel("Stock X price")
34 plt.title("Candlestick for Stock X mm/dd/yyyy")
35 plt.xlim([0, 60])
36 plt.ylim([0, 35])
37 plt.show()

```

will produce

```

1 ((0, 15), 10, 5, 'red')
2 ((10, 10), 10, 4, 'green')
3 ((20, 22), 10, 1, 'green')
4 ((30, 15), 10, 1, 'green')
5 ((40, 12), 10, 14, 'red')
6 ((50, 5), 10, 25, 'green')

```

and the plot.

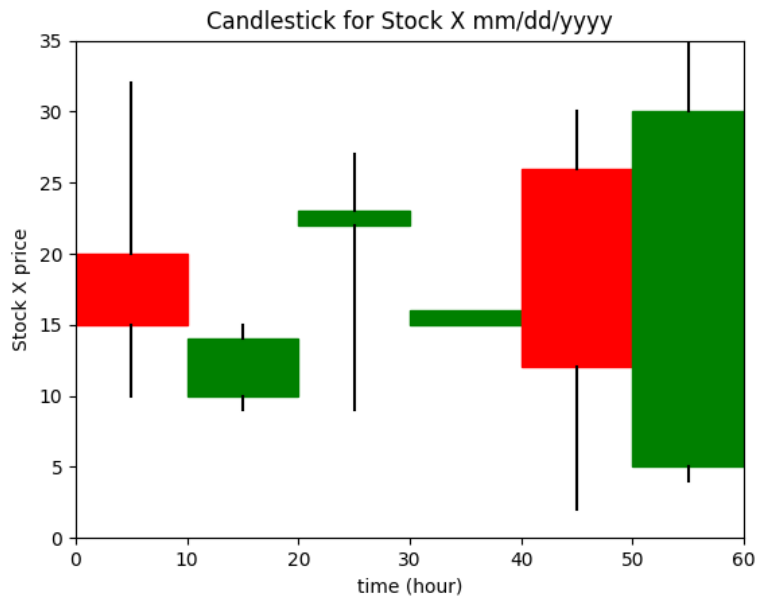


Figure 2: Six candlesticks.

Deliverables for Programming Problem 3

- Complete the function `make` that returns three things: the data for the rectangle, the top line, and the bottom line.
- **Hint:** If you see the output produced by printing `candel_box`, notice how the value of `start` changes for each `candel_box`-this is what makes each candlebox shift in the plot so that each rectangle starts a specific distance after the previous one so that all of them look separated for easy visualization. Specifically, `start` serves as the x-coordinate for the left edge of the rectangle, but its value changes incrementally for each candlestick. The x positions are used to separate candlesticks horizontally and y positions represent the financial values which is (open, close, max, min).
- Keep in mind that after you have tested your code, you must comment out the `'import matplotlib'` and the plotting code given under the `__main__` before submitting to the Autograder. Autograder can not draw graphical plots on the web browser so it will return an error if you do not comment out the plotting code/`import matplotlib`.

Problem 4: Permutations

In this problem, you'll take a list of numbers, then return the permutation. A permutation of a list π is a list $[\pi_0, \pi_1, \dots, \pi_{\text{len}(\pi)-1}]$ of all possible arrangements. The order of elements in the list is unimportant. Here is a run:

```
1 print(permutation([1]))
2 print(permutation([1,2]))
3 print(permutation([1,2,3]))
4 print(permutation([1,2,3,4]))
```

produces:

```
1 [[1]]
2 [[1, 2], [2, 1]]
3 [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
4 [[1, 2, 3, 4], [1, 2, 4, 3], [1, 3, 2, 4], [1, 3, 4, 2], [1, 4, 2, 3], ↵
   [1, 4, 3, 2], [2, 1, 3, 4], [2, 1, 4, 3], [2, 3, 1, 4], [2, 3, 4, ↵
   1], [2, 4, 1, 3], [2, 4, 3, 1], [3, 1, 2, 4], [3, 1, 4, 2], [3, 2, ↵
   1, 4], [3, 2, 4, 1], [3, 4, 1, 2], [3, 4, 2, 1], [4, 1, 2, 3], [4, ↵
   1, 3, 2], [4, 2, 1, 3], [4, 2, 3, 1], [4, 3, 1, 2], [4, 3, 2, 1]]
```

Deliverables for Programming Problem 4

- Complete the function **however** you'd like.
- You **cannot** use any tools or modules—just simple programming.
- My solution uses comprehension and recursion.

5: Complex Numbers

We have implemented a class to represent complex numbers in the starter code. Most functions have been completed for you. Your task is to complete the overloading of power `__pow()` which allows for overloading of exponential and division `__truediv()`. When completed you should be able to run the mandelbrot code. We've also included the code for adding integer and complex numbers (please visit the starter code). We interrogate the types of values to determine when this occurs. We first call `__add()` then `__radd()` to achieve the addition of an integer with a complex number.

Remember to comment out the following plotting code and also the import of `matplotlib` before submitting to the Autograder.

```
1 w = CN(1,2)
2 x = CN(2,1)
3
4 y = complex(1,2)
5 z = complex(2,1)
6
7 for i in range(5):
8     print(i, w**i, y**i)
9
10 print(w/x, y/z)
```

produces:

```
1 0 1 (1+0j)
2 1 (1 + 2j) (1+2j)
3 2 (-3 + 4j) (-3+4j)
4 3 (-11 - 2j) (-11-2j)
5 4 (-7 - 24j) (-7-24j)
6 (0.7999999999999998 + 0.5999999999999999j) (0.8+0.6j)
```

If the complex number class is correct, then you'll have this image:

Deliverables for Programming Problem 5

- Complete the functions `__pow()` and `__truediv()`.
- Remember to comment out the mandelbrot plotting code and also the import of `matplotlib` before submitting to the Autograder.

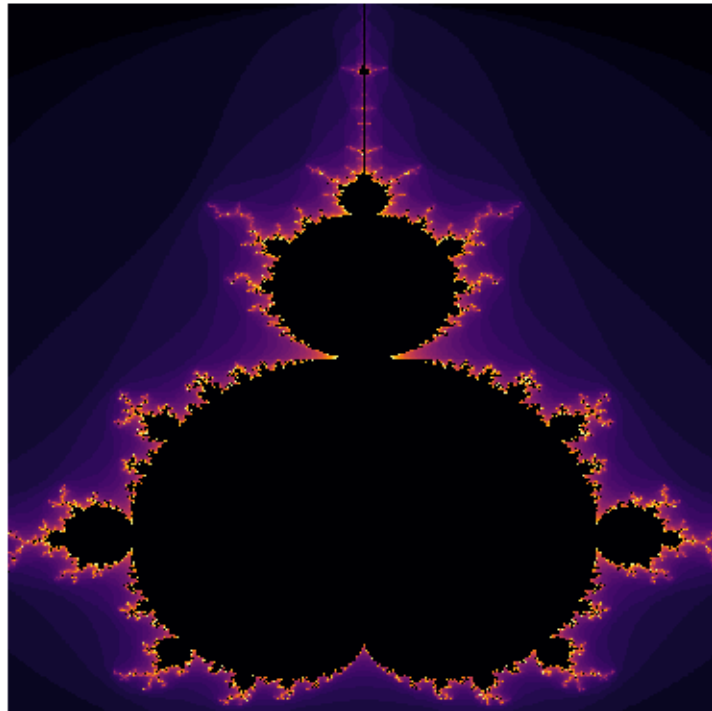


Figure 3: Mandelbrot Set Visualized with complex numbers.

6: Fraction Class

For this problem, you'll be given *some* of the methods of a fraction class. You are responsible for building the required functionality. We define the Fraction class as an ordered pair of positive integers x, y . The first is called the numerator and the second the denominator. When we create an instance, the fraction is **reduced**, *i.e.*, common factors are removed. For example,

$$(2 \times 3 \times 5, 2 \times 3 \times 7) \rightarrow (5, 7) \quad (13)$$

We have only two operations: addition $+$ and multiplication $*$. We also rewrite `__str()` to make the display (x/y) .

Note that when we print the instance (object of a class), the `__str__` function will be called automatically, and instance of the class is returned, not a value. So, x , y , z , and a are all instances of the fraction class and so when we print them, the `__str__` function is called, that returns the representation in the format: numerator/denominator.

When run, the following code:

```
1 x = fraction(2*3*4, 4*3*5)
2 y = fraction(2*7, -7*2)
3 z = fraction(-13, -14)
4 a = fraction(-13*2*7, 14)
```

```

5 print(x,y,z,a)
6 print(f"{x} + {y} == {x + y}")
7 print(f"{x}*{y} == {x * y}")
8 b,c = fraction(1,2),fraction(3,5)
9 print(f"{b} + {c} == {b + c}")

```

has output:

```

1 (2/5) (1/-1) (-13/-14) (-13/1)
2 (2/5) + (1/-1) == (3/-5)
3 (2/5) * (1/-1) == (2/-5)
4 (1/2) + (3/5) == (11/10)

```

The fraction is always in a reduced form. We can use the greatest common divisor algorithm (https://en.wikipedia.org/wiki/Euclidean_algorithm):

```

1 def gcd(a,b):
2     while b != 0:
3         a,b = b,a % b
4     return a
5
6 print(gcd(11*7*5,121*7*3))
7 print(gcd(2*3*5,7*11*13))
8 print(gcd(7*11*13,7*11*13), 7*11*13)

```

producing:

```

1 77
2 1
3 1001 1001

```

Deliverables Problem 6

- Complete the functions inside the Fraction class.
- In class we looked at Euclid's Algorithm for greatest common divisor https://en.wikipedia.org/wiki/Euclidean_algorithm. This is a useful algorithm to reduce the fractions, but you're free to do it as you'd like.
- For this initial attempt, do not consider division by zero and ignore the case when denominator is 0 in a fraction.

Problem 7: Central Dogma, DNA to RNA to Protein

The central dogma in biology is that DNA → RNA → protein. If you want, you can read more about it at https://en.wikipedia.org/wiki/Central_dogma_of_molecular_biology. In this problem you will read in a two files: The first file (amino_acids.txt) contains mapping of codons (a triplet of three DNA bases is called as **codon**) , by mapping we mean a rule that tells which specific codons will convert into a specific amino acid, and the second file (DNA.txt) contains a DNA sequence. The first file will help you to understand the mappings, and then by using the mappings, we will convert the sequence from the second file (DNA.txt) into amino acids. In essence, we will write a program to convert the DNA into protein as living organisms do! (a protein is a sequence of amino acids)

To start with, the contents of amino_acids.txt are shown below in the table.

Name	Abr.	Codons
Isoleucine	I	ATT, ATC, ATA
Leucine	L	CTT, CTC, CTA, CTG, TTA, TTG
Valine	V	GTT, GTC, GTA, GTG
Phenylalanine	F	TTT, TTC
Methionine	M	ATG
Cysteine	C	TGT, TGC
Alanine	A	GCT, GCC, GCA, GCG
Glycine	G	GGT, GGC, GGA, GGG
Proline	P	CCT, CCC, CCA, CCG
Threonine	T	ACT, ACC, ACA, ACG
Serine	S	TCT, TCC, TCA, TCG, AGT, AGC
Tyrosine	Y	TAT, TAC
Tryptophan	W	TGG
Glutamine	Q	CAA, CAG
Asparagine	N	AAT, AAC
Histidine	H	CAT, CAC
Glutamic acid	E	GAA, GAG
Aspartic acid	D	GAT, GAC
Lysine	K	AAA, AAG
Arginine	R	CGT, CGC, CGA, CGG, AGA, AGG
Stop codons	-	TAA, TAG, TGA

The first column is the full name of the amino acid. The second column is the abbreviation as one letter initial (for the same amino acid). For the Stop_codons, we use a dash. The rightmost column are what three letters of DNA are used to make the amino acid. The amino acid Arginine has an abbreviation R. There are six codon (three bases of DNA) that code for Arginine: CGT, CGC, CGA, CGG, AGA, AGG, as can be seen in the table.

About DNA.txt: It's basically a FASTA file. Please don't be confused by the name-it's just a text file that follows certain rules, hence the special name. It has two parts: a header (information about the DNA sequence that it contains) and the DNA sequence itself. Here's the one you'll be using (don't copy them from here, we have already pushed both files to your repositories).

```
>HSLTH1 Human theta 1-globin gene
CCACTGCACTCACCGCACCCGGCCAATTTTGTGTT
TTTAGTAGAGACTAAATACCATATAGTGAACACCTA
AGACGGGGGGCCTTGGATCCAGGGCGATTGAGAGG
GCCCCGGTCGGAGCTGTCGGAGATTGAGCGCGCGC
GGTCCCGGGATCTCCGACGAGGCCCTGGACCCCCG
GGCGGCGAAGCTGCGGCGCGGCGCCCCCTGGAGGC
CGCGGGACCCCTGGCCGGTCCGCGCAGGCGCAGCG
GGGTCGCAGGGCGCGGCGGGTTCAGCGCGGGGAT
GGCGCTGTCCGCGGAGGACCGGGCGCTGGTGCGCG
CCCTGTGGAAGAAGCTGGGCAGCAACGTCGGCGTCT
ACACGACAGAGGCCCTGGAAAGGTGCGGCAGGCTG
GGCGCCCCCGCCCCAGGGGCCCTCCCTCCCCAAG
CCCCCGGACGCGCCTCACCCACGTTCTCTCGCAG
GACCTTCCTGGCTTTCCCGCCACGAAGACCTACTT
CTCCACCTGGACCTGAGCCCCGGCTCCTCACAAGT
CAGAGCCCACGGCCAGAAGGTGGCGGACGCGCTGA
GCCTCGCCGTGGAGCGCCTGGACGACCTACCCAC
GCGCTGTCCGCGCTGAGCCACCTGCACGCGTGCCA
GCTGCGAGTGGACCCGGCCAGCTTCCAGGTGAGCG
GCTGCCGTGCTGGGCCCCTGTCCCGGGAGGGCCC
CGGCGGGGTGGGTGCGGGGGCGTGCGGGGCGGG
TGCAGGCGAGTGAGCCTTGAGCGCTCGCCGCAGCT
CCTGGGCCACTGCCTGCTGGTAACCCTCGCCCGGCA
CTACCCCGGAGACTTCAGCCCCGCGTGCAGGCGTC
GCTGGACAAGTTCCTGAGCCACGTTATCTCGGCGCT
GGTTTCCGAGTACCGCTGAACTGTGGGTGGGTGGCC
GCGGGATCCCCAGGCGACCTTCCCGTGTTTGAGTA
AAGCCTCTCCCAGGAGCAGCCTTCTTGCCGTGCTCT
CTCGAGGTCAGGACGCGAGAGGAAGGCGC
```

Though not necessary but if you want, you can read about this gene here: <https://pubmed.ncbi.nlm.nih.gov/3422341/>. In the file, the first line describes the sequence providing the

name and other attributes. The remaining lines is the DNA sequence (ignore all whitespace), all lines after the header are part of the same sequence so there are no line breaks (ignore whitespaces) in the sequence.

Example: how to translate DNA into a protein

To convert from DNA to protein, we use a sequence of codons. We'll bold the protein when its translated.

Let's look at the first twelve bases: CCACTGCACTCA. Every three bases uniquely determine an amino acid.

1. Start with the first codon CCA, CCACTGCACTCA.
2. Looking at the first file we see: Proline, P, CCT, CCC, CCA, CCG. This means we can rewrite CCA as P.
3. Looking at the next codon CTG, **P**CTGCACTCA
4. We find it matches Leucine, L, CTT, CTC, CTA, CTG, TTA, TTG. So our protein is PL.
5. The next three are CAC **PL**CACTCA.
6. The table has Histidine, H, CAT, CAC.
7. The final three are TCA. **PLH**TCA
8. This matches Serine, S, TCT, TCC, TCA, TCG, AGT, AGC.
9. The protein is **PLHS**.

If you are at the end and only have two bases, you cannot match a protein, so you ignore them. Suppose we had CCAC. We know CCA is P. Then we only have C left. We ignore it.

How to approach this problem

Our main task is to take the DNA (by reading DNA.txt) and produce a string of single letters (as shown in above example, points 1-9) that reflect the encoding. Here is one way to solve this (you should implement how you feel most comfortably)

1. First you'll read in the table from amino_acids.txt, and create a dictionary whose entries are:

$$aa_d = \{(c_0, c_1, \dots, c_n) : [name, letter], \dots\}$$

where c_i is a three letter codon, *name* is the full name of the amino acid, and *letter* is the single letter for the amino acid. Make sure you follow the format of keys and values exactly as shown here. The function get_amino_acid() reads the amino_acids.txt file and returns a dictionary. The dictionary is also shown below in the code listing.

2. Read in the DNA sequence, the function `get_DNA()` reads the `DNA.txt` file and returns a faste data structure [header, DNA] (FASTA data structure) where header is the first line of the file `DNA.txt` and DNA is the DNA sequence (the sequence of A,T,G,C after the first line) (ignoring any whitespace). The read-in FASTA sequence is also shown below in the code listing.

3. The function `translate()` takes a FASTA data structure (that you created in step-2) and returns a string that is the translation using the dictionary (that you created in step-1). We can do a simple print to see whether our translation is the same as actual. An example of `translate()` function is also shown below in the code listing (you are encouraged to cross-check via pen and paper).

This code creates the dictionary and FASTA file (as a list), translates, and validates:

```
1 print("Dictionary")
2 print(aa_d)
3 print("FASTA file")
4 print(DNA_d)
5 print("Translations match:", str(protein == actual))
6
7 #should return "PLHS"
8 print(translate(["nothing", "CCACTGCACTCA"]))
9
10 #should return "D-"
11 print(translate(["nothing", "GACTAA"]))
```

has output:

```
1 Dictionary
2 {( 'ATT', 'ATC', 'ATA'): [ 'Isoleucine', 'I'], ( 'CTT', 'CTC', 'CTA', '←
   CTG', 'TTA', 'TTG'): [ 'Leucine', 'L'], ( 'GTT', 'GTC', 'GTA', 'GTG')←
   : [ 'Valine', 'V'], ( 'TTT', 'TTC'): [ 'Phenylalanine', 'F'], ( 'ATG', )←
   : [ 'Methionine', 'M'], ( 'TGT', 'TGC'): [ 'CYSteine', 'C'], ( 'GCT', '←
   GCC', 'GCA', 'GCG'): [ 'Alanine', 'A'], ( 'GGT', 'GGC', 'GGA', 'GGG')←
   : [ 'Glycine', 'G'], ( 'CCT', 'CCC', 'CCA', 'CCG'): [ 'Proline', 'P'],←
   ( 'ACT', 'ACC', 'ACA', 'ACG'): [ 'Threonine', 'T'], ( 'TCT', 'TCC', '←
   TCA', 'TCG', 'AGT', 'AGC'): [ 'Serine', 'S'], ( 'TAT', 'TAC'): [ '←
   Tyrosine', 'Y'], ( 'TGG', ): [ 'Tryptophan', 'W'], ( 'CAA', 'CAG'): [ '←
   Glutamine', 'Q'], ( 'AAT', 'AAC'): [ 'Asparagine', 'N'], ( 'CAT', 'CAC←
   '): [ 'Histidine', 'H'], ( 'GAA', 'GAG'): [ 'Glutamic_acid', 'E'], ( '←
   GAT', 'GAC'): [ 'AsparTic acid', 'D'], ( 'AAA', 'AAG'): [ 'Lysine', 'K←
   '], ( 'CGT', 'CGC', 'CGA', 'CGG', 'AGA', 'AGG'): [ 'Arginine', 'R'], ←
   ( 'TAA', 'TAG', 'TGA'): [ 'Stop_codons', '-']}]
3 FASTA file
4 [ '>HSLTH1 Human theta 1-globin gene', '←
   CCACTGCACTCACCGCACCCGGCCAATTTTTGTGTTTTTAGT
5 AGAGACTAAATACCATATAGTGAACACCTAAGACGGGGGGC
```

```

6 CTTGGATCCAGGGCGATTTCAGAGGGCCCCGGTCGGAGCTGT
7 CGGAGATTGAGCGCGCGGGTCCCGGGATCTCCGACGAGGC
8 CCTGGACCCCCGGGCGGCGAAGCTGCGGCGGGCGCCCCCT
9 GGAGGCCGCGGGACCCCTGGCCGGTCCGCGCAGGCGCAGCG
10 GGGTCGCAGGGCGCGGCGGGTTCCAGCGCGGGGATGGCGCT
11 GTCCGCGGAGGACCGGGCGCTGGTGC GCGCCCTGTGGAAGA
12 AGCTGGGCAGCAACGTCGGCGTCTACACGACAGAGGCCCTG
13 GAAAGGTGCGGCAGGCTGGGCGCCCCCGCCCCAGGGGCCC
14 TCCCTCCCCAAGCCCCCGGACGCGCCTCACCCACGTTCTCTC
15 TCGCAGGACCTTCCTGGCTTTCCCCGCCACGAAGACCTACTT
16 CTCCACCTGGACCTGAGCCCCGGCTCCTCACAAGTCAGAGC
17 CCACGGCCAGAAGGTGGCGGACGCGCTGAGCCTCGCCGTGG
18 AGCGCCTGGACGACCTACCCACGCGCTGTCCGCGCTGAGC
19 CACCTGCACGCGTGCCAGCTGCGAGTGGAACCGGCCAGCTT
20 CCAGGTGAGCGGCTGCCGTGCTGGGCCCCCTGTCCCCGGGAG
21 GGCCCCGGCGGGGTGGGTGCGGGGGGCGTGCGGGGCGGGT
22 GCAGGCGAGTGAGCCTTGAGCGCTCGCCGAGCTCCTGGGC
23 CACTGCCTGCTGTTAACCTCGCCCGGCACTACCCCGGAGAC
24 TTCAGCCCCGCGCTGCAGGCGTGCCTGGACAAGTTCCTGAGC
25 CACGTTATCTCGGCGCTGGTTTCCGAGTACCGCTGAACTGTG
26 GGTGGGTGGCCGCGGGATCCCCAGGCGACCTTCCCCGTGTTTG
27 AGTAAAGCCTCTCCAGGAGCAGCCTTCTTGCCGTGCTCTCTC
28 GAGGTCAGGACGCGAGAGGAAGGCGC']
29 Translations match: True
30 PLHS
31 D-

```

Deliverables Problem 7

- Carefully read the instructions in the starter file a8.py. You may not be able to run it directly in VSC, so follow the instructions in the starter code.
- Complete the functions `get_DNA()`, `get_amino_acids()` and `translate()` as per the specifications.
- You are allowed to use `replace` for space and `"{"`. Other uses of it will actually be more difficult.
- For this problem, using `pandas` makes the autograder a little crazy—so we're eschewing `dataframes`. Furthermore, everything is text.

Problem 8: Approximating the Derivative

Calculus is used in virtually every field and is fundamental to understanding AI. In this problem we approximate the derivate which is instantaneous change. We'll focus on univariate functions initially:

$$f'(x) = \lim_{\epsilon \rightarrow 0} \frac{f(x + \epsilon) - f(x)}{2\epsilon} \quad (14)$$

This is really a λ function, since we're given a function f and a value ϵ and returning a function f' that has x as an argument. We can approximate $f'(x)$ shown here:

$$f'(x) \approx \frac{f(x + \epsilon) - f(x - \epsilon)}{2\epsilon} \quad (15)$$

We will write a function called derivative that takes a function and epsilon and returns a lambda function. for example,

$$f(x) = x^2 - 3x \quad (16)$$

$$f'(x) = 2x - 3 \quad (17)$$

$$f'(2) = 4 - 3 = 1 \quad (18)$$

```
1 def derivative(f, epsilon):
2     pass
3
4 def f(x):
5     return x**2 - 3*x
6
7 data = 2
8 epsilon = 10e-8
9 print((derivative(f, epsilon)(data)))
10 f_prime = derivative((lambda x: x**2 - 3*x), epsilon)
11 print(f_prime(data))
```

has outputs:

```
1 0.9999999961429751
2 0.9999999961429751
```

Assume you have a real-valued function $f(x)$ and you want to find an optimal value; namely, you want to find for $x \in [a, b]$ the value that makes $f(x)$ the smallest. While we can't develop the full explanation here, we show that if you have an approximation x_i , then this rule will usually work:

$$x_{i+1} \leftarrow x_i - \eta f'(x_i) \quad (19)$$

The $0 < \eta < 1$ is the *learning rate*, and this is multiplied by the derivative at that point. Intuitively, you're moving down toward the smaller value, the derivative indicates by how much. Here's a

simple example of using AI to find the mean. Assume we have a list of numbers $[x_1, x_2, \dots, x_N]$. We know how to analytically find the mean, but we can use AI and the rule above:

$$x_{i+1} \leftarrow x_i - \eta f'(x_i) \quad (20)$$

$$f(x) = \left(\frac{1}{2}\right) \sum_{i=1}^n (x_i - x)^2 \quad (21)$$

The $(1/2)$ makes the problem easier and can be ignored. Using our computational approximation of derivative, we can write the rule as:

```

1  def update(w, data):
2      eta = .2
3      return w - eta*(derivative(lambda x: residuals(data, x), 0.00001) ←
                          (w))

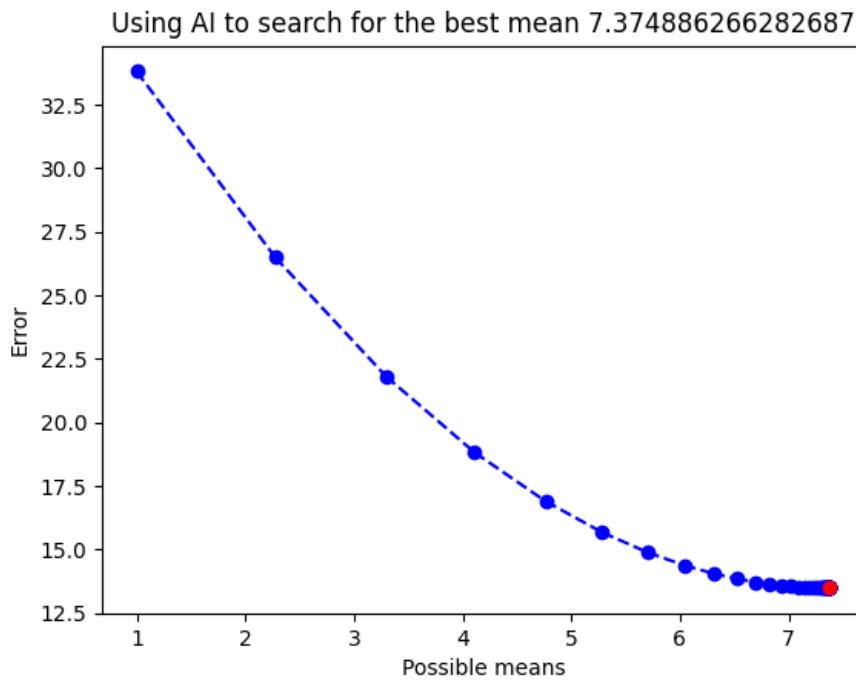
```

We used λ to allow us to send the list of numbers as an additional argument. It can be ignored. But observe we're using the rule. The code below

```

1  N = 50
2  x = np.linspace(1, 14, 100)
3  gm = np.zeros(N)
4  r = np.zeros(N)
5  def mean(lst):
6      s_ = 0
7      N = len(lst)
8      for i in range(N):
9          s_ += lst[i]
10     m_ = round(s_/N, 2)
11     return m_
12
13     #this is eq. 30...it's also gives error in our approximation
14     def residuals(lst, m):
15         s_ = 0
16         N = len(lst)
17         for i in range(N):
18             s_ += (lst[i] - m)**2
19         m_ = (1/2)*(s_/N)
20         return m_
21     data = [1, 1, 2, 6, 10, 12, 13, 14]
22
23     #update rule
24     def update(w, data):
25         eta = .2
26         epsilon = 0.00001
27         return w - eta*(derivative(lambda x: residuals(data, x), epsilon) ←
                                (w))
28

```



```

29     m_ = mean(data)
30
31     #successively applying the update rule
32     fmean = 1
33     for i in range(N):
34         gm[i] = fmean
35         r[i] = residuals(data,fmean)
36         #print(fmean,residuals(data,fmean))
37         fmean = update(fmean,data)
38
39     print(gm[-1])
40     print(m_)
41     plt.plot(gm,r,'bo')
42     for i in range(1,N):
43         plt.plot([gm[i-1],gm[i]],[r[i-1],r[i]],'b--')
44     plt.plot(m_,residuals(data,m_),'ro')
45     plt.xlabel("Possible means")
46     plt.ylabel("Error")
47     plt.title(f"Using AI to search for the best mean {gm[-1]}")
48     plt.show()

```

produces Fig. as successive approximations to the red value which is the mean.

Deliverables for Programming Problem 8

- Complete the derivate function